

Week 2 Assignment

Build Your Personal Portfolio Website

Apply HTML · CSS · JavaScript · GitHub · Vercel

■ 12 July 2026

■ Due: 20 July 2026

■ Live Deployment Required

■ Submission Deadline

Monday, 20 July 2026 at 11:59 PM · No late submissions accepted

What You Are Building

This week you learned how the internet works, how HTML creates structure, how CSS makes pages beautiful, how JavaScript adds interactivity, and how GitHub and Vercel publish your work to the world. Now you apply all of that knowledge by building your own personal portfolio website from scratch and deploying it live on the internet.

■ index.html

Structure and content of every page
Semantic tags, sections, navigation

■ styles.css

Design and layout
Colors, Flexbox, responsive design

■ script.js

Interactivity and live data
DOM, events, fetch from API



Part 1 — HTML Structure

Build the skeleton using semantic HTML tags

HTML is the foundation of every webpage. It defines what content exists and how it is organised. You will use semantic tags that clearly describe each section of your page so browsers and search engines understand your content.

1

Create a folder called my portfolio on your Desktop. Inside it, create three files: index.html, styles.css, and script.js. Open the folder in VS Code.

2

Write the full HTML boilerplate including DOCTYPE declaration, the html element with lang attribute, head section with charset, viewport meta tag, title, and link to your CSS file, and body element with script tag at the very bottom.

3

Add a header element containing a nav element. Inside nav, put your name or logo on the left side and an unordered list of four navigation links pointing to: #about, #skills, #projects, and #contact.

4

Add a hero section inside the header element, below the nav. Include your name as an h2 heading, a subtitle paragraph describing who you are, and a profile image using the img tag with an alt attribute.

5

Add a main element after the header. Inside main, create three sections with unique id attributes: id="about" with a bio paragraph, id="skills" with at least four skill div elements each having class="skill-card", and id="projects" with at least two project div elements each having class="project-card" containing an h3 heading and a description paragraph.

6

Add a footer element with id="contact" containing copyright text using the © entity and your email as a clickable mailto: link.

7

Use only semantic HTML tags throughout: header, nav, main, section, footer. Avoid using div elements for page structure.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Your Name | Portfolio</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <header> ... nav + hero ... </header>
    <main> ... sections ... </main>
    <footer> ... copyright + email ... </footer>
    <script src="script.js"></script>
  </body>
</html>
```



Part 2 — CSS Styling

Make it beautiful with colors, Flexbox, and responsive design

CSS transforms a plain HTML page into a visually polished website. You will use Flexbox to create professional layouts and media queries to make your page look great on phones.

1

Add a CSS reset at the very top of styles.css using the universal selector star with margin 0, padding 0, and box-sizing border-box. Then set a global font-family, background-color, and text color on the body element.

2

Style the header element with a linear-gradient background using two colors of your own choice. Make the text inside white. Add generous padding for breathing room.

3

Apply Flexbox to the nav element: set display to flex, justify-content to space-between to push the logo left and links right, and align-items to center. Remove bullet points from the navigation list using list-style: none. Remove link underlines using text-decoration: none.

4

Make the hero section text-align center. Use max-width with margin 0 auto to center the content. Give the profile image a circular shape using border-radius 50 percent and add a white border.

5

Style the skills container with display flex, flex-wrap wrap, gap, and justify-content center so skill cards sit side by side and wrap naturally to the next line.

6

Style each skill card and project card with white background, padding, border-radius for rounded corners, and box-shadow for a floating card effect.

7

Add at least one hover effect using the colon hover pseudo-class on a card or button. The element should visually respond when the mouse moves over it.

8

Add a media query using @media with max-width 768px at the bottom of your CSS file. Inside it, change the nav and skills container flex-direction to column so everything stacks vertically on mobile screens.

9

Style the footer with a dark background such as #333333, white text, centered alignment, and a top margin to separate it from the content above.

■ TIP Choosing Colors

Go to Google and search "color picker". Select any color you like and copy the hex code that starts with #. Paste it directly into your CSS. Great palettes: blue + purple, orange + dark navy, green + white. Stick to two or three colors for a professional look.



Part 3 — JavaScript Interactivity

Bring the page to life with DOM, events, and live API data

JavaScript is what makes a webpage respond to users. You will add a dark mode toggle button, respond to click events, and use the fetch function to pull live data from the internet and display it on your page.

1

Make sure script.js is linked at the BOTTOM of the body element in index.html, just before the closing body tag. JavaScript must load after HTML so it can find the elements it needs.

2

Add a dark mode toggle button in your navigation with an id attribute. Write CSS rules for a `body.dark-mode` class that changes background to dark and text to light colors.

3

In `script.js`, find the dark mode button using `document.getElementById`. Attach a click event listener using `addEventListener`. When clicked, toggle the dark-mode class on `document.body` using `classList.toggle`. Also update the button text between a moon emoji and a sun emoji.

4

Find the button using `getElementById` or `querySelector`. Never write `onclick` directly inside HTML tags. All JavaScript logic stays in `script.js`.

5

Add a second button and a paragraph element for displaying API data. Use `fetch` to call one of the free APIs listed below. Show "Loading..." while waiting for data, then display the response. Add a catch handler to show a friendly error message if the request fails.

Choose one free API for your fetch feature:

Option A Random Joke:

```
fetch("https://official-joke-api.appspot.com/random_joke")
  .then(res => res.json())
  .then(data => { /* use data.setup and data.punchline */ })
```

Option B Random Quote:

```
fetch("https://api.quotable.io/random")
  .then(res => res.json())
  .then(data => { /* use data.content and data.author */ })
```

Option C Random Activity:

```
fetch("https://www.boredapi.com/api/activity")
  .then(res => res.json())
  .then(data => { /* use data.activity */ })
```



Part 4 — Push to GitHub

Save your code safely and professionally online

GitHub is where developers store and share their code. Employers look at GitHub profiles. Pushing your code here is also required for Vercel to deploy it automatically.

1

Create a free account on `github.com` using your real name as your username. This is your professional developer identity that employers and collaborators will see.

2

Create a new repository called `my-portfolio`. Set it to Public so others can see it. Do not check "Initialize with README" as we will push our own files.

3

Open the VS Code terminal by pressing Ctrl and the backtick key together. Run these five commands one at a time, pressing Enter after each one. Replace YOUR-USERNAME with your actual GitHub username.

4

After pushing, refresh your GitHub page. You should see all three files: index.html, styles.css, and script.js. Take a screenshot as proof of your submission.

```
Step 1 git init
Step 2 git add .
Step 3 git commit -m "my portfolio website"
Step 4 git branch -M main
Step 5 git remote add origin https://github.com/YOUR-USERNAME/my-portfolio.git
Step 6 git push -u origin main
```



Part 5 — Deploy to Vercel

Make your portfolio live on the internet for the world to see

Vercel gives your website a real public URL. Anyone in the world can open it. This is the link you submit, share on your CV, and show to employers.

1

Go to vercel.com and click Sign Up. Choose the option "Continue with GitHub" so Vercel connects directly to your GitHub account. This takes only one click.

2

Click "Add New Project" on the Vercel dashboard. You will see a list of your GitHub repositories. Find "my-portfolio" and click Import.

3

On the configuration screen, set the Framework Preset to "Other". Leave all other settings empty. Click the Deploy button.

4

Wait 30 to 60 seconds. When Vercel says "Congratulations!" you will receive a URL such as my-portfolio-yourname.vercel.app. Click it and confirm your portfolio opens correctly.

5

Open the Vercel URL on your mobile phone. Take a screenshot. Check that it looks good on a small screen. Test the dark mode button and the fetch button on mobile as well.

■ AUTO DEPLOYMENT How It Works After This

From now on, every time you push new code to GitHub using `git add .` then `git commit` then `git push`, Vercel will automatically update your live website within 30 seconds. No extra steps needed. This is how professional developers ship code.



Part B — Bonus Challenges

Optional tasks for students who want to go further

B1

Add smooth scroll behaviour so that clicking a navigation link scrolls the page smoothly to that section instead of jumping to it instantly.

B2

Add a second fetch call that uses a completely different API from the one you chose in Part 3 and displays its data on a separate part of the page.

B3

Add a card hover animation using CSS transform translateY with a negative value so cards appear to lift up when the mouse hovers over them. Combine this with CSS transition for a smooth effect.

B4

Add a contact form using the HTML form element with input fields for name and email and a textarea for a message. Style it with CSS so it matches the rest of the page.

B5

Connect a custom domain name from namecheap.com to your Vercel project. A domain costs approximately one thousand Pakistani rupees per year.

Marking Criteria

Area	What the Trainer Will Check	Marks
HTML Structure	Correct boilerplate, semantic tags, all required sections present	20
CSS Styling	Gradient header, Flexbox layout, card styling, hover effect present	20
Responsive Design	Media query exists, page adapts correctly on mobile screen	15
JavaScript	Dark mode works correctly, addEventListener used, fetch displays live data	20
GitHub	Public repository exists with all three files visible	10
Vercel Deployment	Submitted URL opens in browser and displays the portfolio	10
Personalisation	Real name, real skills, real projects. No placeholder text remaining	5
TOTAL		100

Pre-Submission Checklist

Go through every item below before you submit. If any item is not ticked, fix it first.

- My portfolio is complete with real personal information. No placeholder text remains.
- The dark mode button works when clicked. The page switches between light and dark.

- The fetch button works and displays live data from an API on the page.
- The page looks correct on mobile. I tested it using Chrome DevTools or my phone.
- All three files are pushed to a PUBLIC GitHub repository.
- The Vercel URL opens correctly in a browser and shows my portfolio.
- I have copied both my GitHub link and my Vercel link ready to paste into the form.

Tips for Success

■ WHEN SOMETHING DOES NOT WORK

Press F12 in Chrome and click the Console tab. Red text shows the error and the exact line number where it happened. Read the error. Google it. This is exactly how professional developers debug every single day.

■ IF CSS CHANGES ARE NOT SHOWING

Make sure you saved the file using Ctrl S. Make sure the link tag in the head section says href="styles.css" with the exact same name as your file. Try a hard refresh in the browser using Ctrl Shift R.

■ IF JAVASCRIPT IS NOT RUNNING

Check that your script tag is at the BOTTOM of the body element, not inside the head section. Then open the Console in F12 and look for red errors. Add console.log("test") as the very first line of script.js to confirm it is loading.

■ IF YOU GET STUCK

Ask in the WhatsApp group. Share your Vercel link or GitHub link so your trainer can see the problem. Include a screenshot of any red error from the Console. Do not wait until the night before the deadline.

■ Submission Deadline

Monday, 20 July 2026 at 11:59 PM · No late submissions accepted

■ Submit Your Assignment

https://docs.google.com/forms/d/e/1FAIpQLSfIV-paYLAqkNCXiBXOIYuQ4lxKacGn00kDDbq5Fh-XisZn_w/viewform?usp=header

Submit your GitHub repository link and your Vercel deployed link using the Google Form above. The form closes automatically at 11:59 PM on 20 July 2026. Submissions received after the deadline will not be accepted.

